Concurrency and Distributed Systems

Week 5 – Part 2: BlockingQueue, Producer-Consumer, and Backpressure

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 2 Roadmap

1. Why Queues Need Capacity
2. BlockingQueue Semantics
3. Producer-Consumer Backpressure
4. Shutdown and Coordination
5. Lab Preparation

Why This Part Matters

Core question

If task submission is easy, what prevents producers from flooding the system?

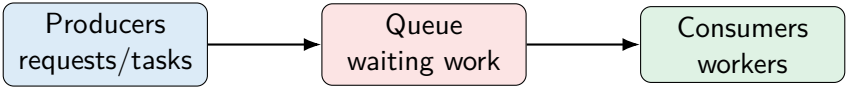
Lecture 05.01 gave us managed task execution. This lecture adds the missing operational contract:

How much waiting work is allowed?

Week 5 principle

An executor without a capacity policy can still become an overload machine.

The Queue Is Part of the System



- Queues absorb short bursts.
- Queues do not create extra processing capacity.
- Queue growth is delayed failure when producers permanently outrun consumers.

Backpressure

Definition

Backpressure is a signal to upstream producers that the downstream system is full or slow.

- Block the producer until space is available.
- Reject work and make the caller handle overload.
- Drop work only when the business rule allows it.
- Slow down callers by running work in the caller's thread.

No signal means no control

Unbounded queues remove the signal until memory or latency fails.

Demo01: Unbounded Queue Risk

```
1 Queue<Integer> queue = new ConcurrentLinkedQueue<>();
2 for (int i = 0; i < items; i++) {
3     queue.add(i);
4 }
5 return queue.size();
```

- `ConcurrentLinkedQueue` is thread-safe.
- Thread-safe does not mean capacity-safe.
- Every producer succeeds immediately.
- Memory becomes the hidden capacity limit.

Unbounded Queue Failure Mode

1. Producers enqueue faster than consumers process.
2. Queue depth grows.
3. Waiting time grows.
4. Latency becomes unacceptable.
5. Memory pressure appears.
6. The system fails after already serving bad latency.

Teaching point

Out-of-memory is the final symptom. The first symptom is usually queueing delay.

Queue Capacity Is a Design Decision

Capacity choice	Consequence
Too small	Rejects or blocks during normal bursts.
Too large	Hides overload and increases latency.
Unbounded	Memory becomes the only limit.
Bounded	Overload becomes explicit and testable.

Rule

Pick capacity from the latency and memory budget, not from convenience.

BlockingQueue Families

- `ArrayBlockingQueue`: bounded, array-backed, predictable capacity.
- `LinkedBlockingQueue`: optionally bounded; dangerous if left unbounded.
- `PriorityBlockingQueue`: priority order; unbounded by default.
- `SynchronousQueue`: no storage; direct handoff from producer to consumer.

For Week 5 exercises

Use `ArrayBlockingQueue` when the requirement is explicit bounded capacity.

BlockingQueue Operations

Operation	Throws	Special value	Blocks
Insert	`add(e)`	`offer(e)`	`put(e)`
Remove	`remove()`	`poll()`	`take()`
Inspect	`element()`	`peek()`	–
Timed	–	`offer(e,t,u)`	`poll(t,u)`

API choice expresses policy

Use `put/take` to wait, `offer/poll` to avoid waiting, and timed variants for bounded waiting.

Demo02: Bounded BlockingQueue

```
1 BlockingQueue<String> queue = new ArrayBlockingQueue<>(1);
2 return queue.offer("first") && !queue.offer("second");
```

- Capacity is 1.
- First insert succeeds.
- Second insert returns `false`.
- The caller sees overload immediately.

Important distinction

`offer()` does not block. It asks: "Can you accept this now?"

Blocking Insert with put

```
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(capacity);

queue.put(task); // waits until space exists
```

- Natural fit when every task must eventually be processed.
- Producer thread becomes part of the backpressure path.
- Must handle `InterruptedException`.

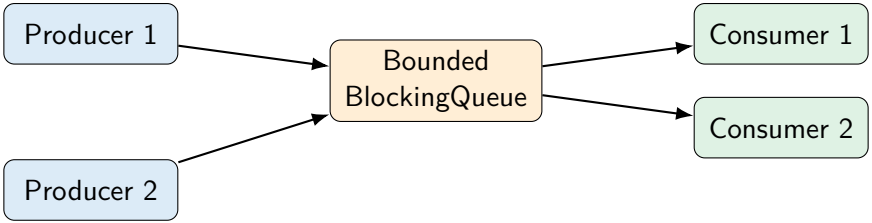
Timed Insert with offer

```
boolean accepted =
    queue.offer(task, 100, TimeUnit.MILLISECONDS);

if (!accepted) {
    rejectRequest();
}
```

- Waits only up to the caller's time budget.
- Useful in request/response systems.
- Turns overload into an explicit branch.

Producer-Consumer Contract



- Producers own input rate.
- Consumers own processing rate.
- The queue exposes mismatch between them.

Demo03: Producer-Consumer Backpressure

```
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(1);

Thread producer = new Thread(() -> {
    queue.put(1);
    queue.put(2); // waits until consumer takes first
});

int first = queue.take();
int second = queue.take();
```

What students should notice

The second `put` cannot complete until the consumer creates space.

Liveness Questions

- Can a producer wait forever?
- Can a consumer wait forever?
- What happens when shutdown starts?
- How does interruption propagate?
- How do we prove the queue is not hiding deadlock?

Week 5 scorecard

Every bounded design must include a liveness note: what can block, and how does it unblock?

Poison Pill Shutdown

```
queue.put(POISON);

while (true) {
    Task task = queue.take();
    if (task == POISON) {
        break;
    }
    task.run();
}
```

- A poison pill is an in-band shutdown signal.
- It works when producers and consumers agree on a sentinel.
- Use one poison pill per independent consumer.

Poison Pill Caveats

- It must not be confused with real work.
- It can be delayed behind earlier queued work.
- With multiple consumers, one pill usually stops only one consumer.
- It is not a substitute for interrupt-aware blocking code.

Rule

Use poison pills for orderly drain-and-stop. Use interruption for cancellation.

Preview: Semaphore

Mental model

A `Semaphore` controls access using permits.

- Good for bounding access to a resource pool.
- Does not store work items.
- Complements queues but does not replace them.

Example

Allow at most 10 concurrent database calls even if the executor has more workers.

Preview: Latch and Barrier

Tool	Purpose
`CountDownLatch`	One-time gate: wait until N events happen.
`CyclicBarrier`	Reusable phase boundary: N threads meet repeatedly.

Placement

These are useful coordination tools, but the core Week 5 exercise is bounded queueing.

What to Measure

- Queue size over time.
- Remaining capacity.
- Number of blocked or rejected submissions.
- Producer wait time.
- Consumer throughput.
- End-to-end latency.

Checkpoint 2

A thread-pooled server should report request count, latency, and overload behavior.

Exercise 1: BoundedBuffer

```
public class Ex1_BoundedBuffer<T> {
    public Ex1_BoundedBuffer(int capacity) { ... }
    public void put(T item) throws InterruptedException { ... }
    public T take() throws InterruptedException { ... }
    public int size() { ... }
    public int remainingCapacity() { ... }
}
```

- Use `ArrayBlockingQueue`.
- Preserve blocking semantics.
- Do not busy-wait.
- Let interruption propagate.

Common Mistakes in Ex1

- Using `ConcurrentLinkedQueue`: thread-safe but unbounded.
- Using `offer()` when the requirement says block.
- Catching `InterruptedException` and ignoring it.
- Returning `null` when the queue is empty instead of blocking.
- Implementing the queue manually with ad hoc locks.

Lab Preparation

- Read `Demo01_UnboundedQueueRisk`.
- Read `Demo02_BoundedBlockingQueue`.
- Read `Demo03_ProducerConsumerBackpressure`.
- Then implement `Ex1_BoundedBuffer`.

Reading prompt

For each demo, identify what happens when the queue is full and what happens when it is empty.

Part 2 Takeaway

Final takeaway

A bounded queue turns overload from a hidden memory problem into an explicit concurrency policy.

Bridge to Part 3

Once we can bound waiting work, the next question becomes:

How many workers should process it?

End of Part 2

Questions?